TaskFlow

Une todo-list web construite avec l'IA, sans framework ni serveur.

Objectif : livrer une application qui marche vraiment, en HTML, CSS et JavaScript natif, en gardant la main sur chaque décision. L'IA propose, le binôme décide.

M **Mark Albanese--Alessandrini**
Bachelor 1 · Ynov Toulouse

M **Mathieu Montjat**
Bachelor 1 · Ynov Toulouse

Base · sprint 1
Créer, afficher, modifier, terminer, supprimer, sauvegarder (localStorage)

Informatique · sprint 2
Filtre par statut, tri, catégories colorées, recherche par mot-clé

Cybersécurité · sprint 3
Connexion, robustesse du mot de passe, jauge, SHA-256, blocage après 3 échecs

Cadrer d'abord, coder ensuite. 21 prompts tracés.

OUTILS ET POURQUOI

CC **Claude Code (dans VS Code)**
Lit tout le dépôt, lance les tests, gère Git. Le fichier CLAUDE.md lui redonne les règles du cours à chaque session.

CD **Claude Design**
Maquette de l'interface, adaptée ensuite à nos règles (pas de CDN, icônes SVG locales).

GH **GitHub + Actions**
Une branche par fonctionnalité, PR, mise en ligne sur GitHub Pages.

DÉMARCHE : 3 ITÉRATIONS SUR LES USER STORIES

PROMPT 4
8 stories, mais catégories et cybersécurité oubliées

PROMPT 7
15 stories, découpées en 3 sprints

PROMPT 9
Critères d'acceptation testables

NOTRE PROMPT · ARCHITECTURE (PROMPT 5)

« Tu es un architecte logiciel senior. [...] Lis CLAUDE.md intégralement. Ne fais aucune supposition sur ce qui n'est pas documenté. Si plusieurs architectures sont envisageables, compare-les. Ne génère aucun code pour le moment. »

RÉPONSE DE L'IA

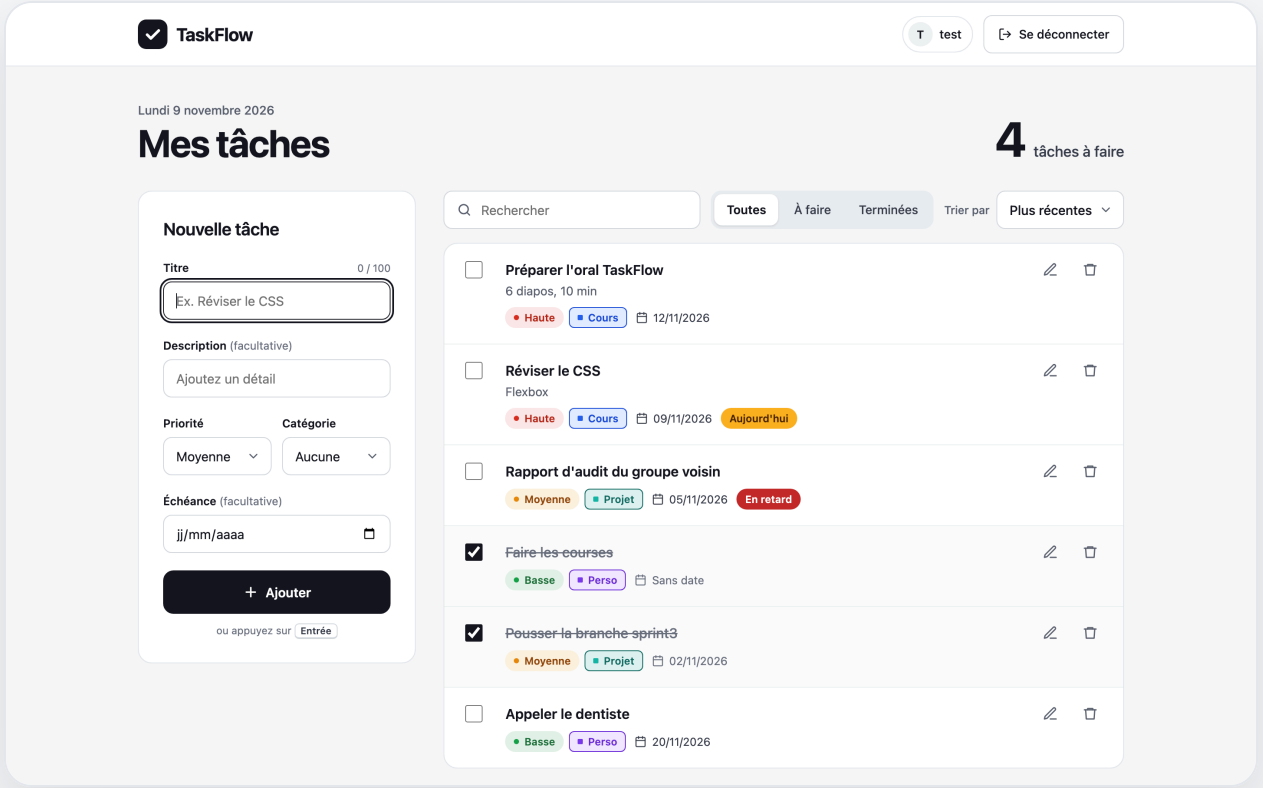
3 architectures comparées. Recommandée : un fichier JS par rôle (`Stockage`, `Auth`, `Taches`, `Interface`), sans modules ES. Plus 9 questions ouvertes au lieu d'inventer les réponses.

NOTRE DÉCISION

Gardé : un fichier par rôle. Rejeté : un seul app.js (conflits entre nous) et les modules ES (ne marchent pas en double-cliquant sur index.html).

3 Résultat obtenu

2 min · Mark



Haute · Moyenne · Basse · Cours · Perso · Projet

Démonstration en direct

1. Premier lancement : mot de passe refusé, puis la jauge passe à **Fort**
2. Ajout d'une tâche, titre vide refusé, `<b>` affiché en texte
3. Cocher, modifier dans la liste, supprimer
4. Filtre « À faire », tri par échéance, recherche « css »
5. F5 : tout est retrouvé
6. 3 mauvais mots de passe : compte bloqué 20 s
7. Outils de développement : seule l'empreinte SHA-256 est stockée

- 16 user stories, toutes livrées
- Pas encore testé à la main sur Firefox et Safari

L'IA s'est trompée. On l'a vu, compris, corrigé.

ERREUR	COMMENT ON L'A VUE · CE QU'ON A FAIT
A supposé que notre filière était IA & Data	Elle s'est fiée au nom du dossier `ia-data`. Corrigé au prompt suivant.
User stories sans catégories ni cybersécurité	Relecture face au cahier des charges : 2 itérations de plus.
Fermer une modification sans l'enregistrer	Choix refusé : la saisie aurait été perdue. Elle est enregistrée d'abord.
Maquette avec Google Fonts et zone de texte	CDN interdit, et la touche Entrée n'ajoutait plus la tâche. Remplacées.
« Compte bloqué » resté affiché après connexion	Trouvé par les tests automatiques avant le commit, puis corrigé.

COMMENT ON A VÉRIFIÉ

106 vérifications dans Chrome

51 tests Node sur les fonctions pures

Chaque story a des critères d'acceptation cochés un par un. Et nous avons relu chaque choix proposé par l'IA.

LIMITES

Les tests sont écrits par l'IA elle-même : ils vérifient ce qu'elle a compris, pas forcément ce que nous voulions.

L'IA aussi nous a corrigés : un mot de passe « test » contredisait notre propre règle de robustesse.

Une connexion côté navigateur ne protège que l'affichage.

Les risques

- localStorage lisible et modifiable par n'importe qui
- SHA-256 rapide et sans sel
- Blocage contourné en vidant les données
- « Compte bloqué » révèle qu'un compte existe

Toutes ces limites sont écrites dans le README.

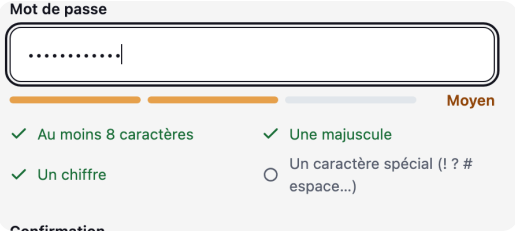
Ne jamais donner à l'IA

- Mots de passe, clés d'API, jetons
- Données personnelles réelles (nom, e-mail, santé)
- Code ou données d'un client sous contrat

Chez nous : comptes de test fictifs, et un bandeau « n'utilisez pas un vrai mot de passe ».

Responsabilité

- Le code livré est le nôtre, pas celui de l'IA
- En entreprise : outils validés, charte IA, revue de code humaine
- L'IA ne fusionne jamais dans `main` : c'est nous qui validons



Ce qu'un vrai site ferait : vérification sur un **serveur**, hachage **lent et salé** (bcrypt, Argon2), même message d'erreur dans tous les cas. Hors de portée sans serveur, donc signalé plutôt que caché.

L'IA va vite. La qualité vient de nos questions.

CE QU'ON A APPRIS

- Un prompt précis (rôle, contraintes, format) change tout
- Faire poser des questions à l'IA avant le code
- Tout garder par écrit : CLAUDE.md, PROMPTS.md

DANS NOTRE FUTUR MÉTIER

- **Développeur** : l'IA écrit, nous relisons et testons
- **Cybersécurité** : le code généré se vérifie comme n'importe quel autre code

CE QU'ON FERAIT AUTREMENT

- Un premier prompt complet dès le départ
- Écrire nous-mêmes une partie des tests
- Tester Firefox et Safari à chaque sprint

> Une erreur bien repérée, comprise et corrigée vaut autant qu'un résultat parfait.
>
> Barème de l'oral, séance 3

Merci. Des questions ?

github.com/M-PROGRAMMATION/taskflow